SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**

Question Count: 5

Code of Conduct

I **M. Meghana (192525435)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.Meghana*

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1.A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Big Data Platform for Website Clicks, Mobile Interactions and Transactions

The platform handles billions of events every day, so it most likely uses a distributed big data architecture**.**

1. Distributed Storage System

- **HDFS** or cloud object storage such as Amazon S3 is likely used for storing huge volumes of data.
- Data is distributed across multiple nodes for scalability and fault tolerance.
- Structured transaction data may be stored in Hive tables or distributed databases.
- Semi-structured clickstream and mobile data can be stored in Parquet/Avro/JSON formats.

2. Data Ingestion

- **Apache Kafka** is likely used to collect website clicks, mobile events and transactions in real time.
- **Apache Flume** or similar ingestion tools may be used for collecting log data.
- Kafka topics can be separated by event type, such as clicks, payments and customer activity.

3. Batch Processing

- **Apache Spark** is likely used for large-scale batch processing.
- **Hive** can provide SQL-based analysis over large datasets.
- Spark can clean, transform and aggregate historical customer and transaction data.

4. Data Organization and Indexing

- Structured data such as orders and payments is stored in tables with defined schemas.
- Semi-structured data such as click logs is stored in flexible formats such as JSON.
- Data is probably partitioned by date, region, customer or event type.
- Columnar formats such as Parquet improve analytical query performance.

5. Stream Processing

- Spark Streaming**,** Apache Flink, or Kafka Streams can process events continuously.
- These tools support real-time dashboards, fraud detection and customer activity monitoring.

6. Scalability and Partitioning

- Kafka partitions distribute incoming events across brokers.
- HDFS distributes files across storage nodes.
- Spark distributes processing across worker nodes.
- Horizontal scaling allows additional nodes to be added when data volume increases.

7. Analytics and Dashboards

- Processed data is loaded into an analytical warehouse or data mart.
- BI tools such as Power BI, Tableau, or Looker connect to the analytics layer.
- Managers can monitor sales, customer behavior, traffic and conversion rates through dashboards.

2.An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

. E-Commerce Platform for Personalization

The e-commerce system requires real-time processing because customer actions must immediately influence recommendations.

1. Event Streaming

- **Apache Kafka** is likely used for collecting product views, cart additions, checkout events and payments.
- Different Kafka topics can represent different customer activities.
- Events are processed continuously rather than waiting for batch jobs.

2. Customer Profile Database

- A distributed NoSQL database such as Cassandra, Mongo DB or H Base may store customer profiles.
- Frequently accessed profiles can be maintained in Redis for fast retrieval.

3. Session and Clickstream Processing

- Kafka receives clickstream events.
- **Flink or Spark Streaming** processes these events.
- Session information can be created using customer ID, device ID and timestamps.
- Recent activity is combined with historical customer behavior.

4. Workflow
The probable workflow is:

Customer Event → Kafka → Stream Processing → Customer Profile → Feature Generation → ML Model → Recommendation → Website/App

- Machine learning models analyze browsing and purchasing patterns.
- Collaborative filtering or recommendation models identify suitable products.
- Real-time features improve personalized recommendations.

5. Caching

- **Redis** is likely used for product information, customer profiles and recommendations.
- Caching reduces database access time and improves response speed.

6. Distributed Query Engine

- **Presto/Trino** or Spark SQL can query large datasets stored in a data lake.
- This helps analysts combine historical and current customer information.

7. Structured and Semi-Structured Data

- Structured order/payment data can be stored in relational tables.
- Browsing and clickstream data can be stored as JSON or Parquet.
- A common customer ID allows both datasets to be joined for analytics.

8. Scalability and Fault Tolerance

- Kafka replication prevents loss of events.
- Distributed databases replicate data across nodes.
- Spark/Flink automatically distributes processing.
- Cloud-based scaling can increase resources during high-traffic periods.

9. KPI Monitoring
Dashboards can display:

- Conversion rate
- Cart abandonment rate
- Sales revenue
- Average order value
- Product views
- Recommendation performance

3.A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Healthcare Big Data System

Healthcare data contains highly sensitive structured, semi-structured and unstructured information. Therefore, a hybrid architecture is likely.

1. Hybrid Database Architecture

- Relational databases such as PostgreSQL/Oracle can store patient IDs, prescriptions and laboratory records.
- **HDFS/S3** can store medical images, documents and historical files.
- NoSQL databases can store wearable sensor and semi-structured data.
- Specialized image/object storage may be used for diagnostic images.

2. Data Integration
Hospitals, laboratories and insurance systems may use:

- **Apache NiFi**
- ETL tools
- APIs
- Healthcare interoperability standards such as **HL7/FHIR**

These systems transform and integrate data from different healthcare providers.

3. Streaming Data
Wearable devices continuously generate heart rate, blood pressure and activity information.

- Kafka can ingest sensor streams.
- Flink or Spark Streaming can process them in real time.
- Alerts can be generated when abnormal readings are detected.

4. Machine Learning
Machine learning can be used for:

- Disease prediction
- Patient risk scoring
- Readmission prediction
- Medical image analysis
- Patient monitoring

5. Privacy and Security
Healthcare data requires strong security controls:

- Encryption at rest and in transit
- Role-Based Access Control
- Authentication and authorization
- Data masking/anonymization
- Audit logs
- Secure APIs

6. Compliance
The platform should follow applicable healthcare privacy and data-protection regulations. Only authorized personnel should access sensitive patient information.

7. Clinical Decision Support
Doctors can receive risk scores, alerts and summarized patient information to support clinical decisions.

8. Population Health
Aggregated and de-identified data can be analysed to identify disease trends, treatment outcomes and population-level health patterns.

4.A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Smart City Big Data Platform

Data from traffic sensors, CCTV systems, weather stations and public transport.

1. Edge Computing

- Edge devices are placed close to sensors.
- They perform initial filtering and processing.
- Only important or summarized data is sent to the central platform.
- This reduces network bandwidth and latency.

2. Distributed Messaging

- **Apache Kafka** or MQTT-based systems can collect continuous sensor events.
- Separate topics can be created for traffic, weather, transport and CCTV metadata.

3. Centralized Storage

- A data lake based on **HDFS or cloud object storage** can store historical data.
- Raw sensor data can be retained for future analysis.
- Processed data can be stored in analytical databases.

4. Time-Series and Geospatial Databases

- Influx DB/Timescale DB can store sensor measurements over time.
- Postgre SQL with Post GIS can support location-based queries.
- Geographic coordinates can be used to identify traffic congestion and accident locations.

5. Real-Time Processing

- Apache Flink or Spark Streaming can analyze traffic and transport events in real time.
- The system can detect congestion and unusual traffic patterns.

6. Congestion Prediction
Historical traffic data is combined with:

- Current traffic conditions
- Weather
- Public transport activity
- Time of day
- Location

Machine learning models can then predict future congestion.

7. Real-Time Analytics

- Real-time analytics provides immediate traffic alerts.
- Batch analytics studies long-term traffic patterns.
- Urban planners can use historical results for road planning and infrastructure development.

8. Security

- Role-based access control
- Encryption
- Secure device authentication
- Network segmentation
- API security
- Audit logging

These measures protect citizen and infrastructure information.

9. Visualization
Tools such as Power BI, Tableau or Grafana can provide:

- Live traffic maps
- Transport status
- Weather information
- Congestion alerts
- Infrastructure monitoring

5.A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations

Healthcare Analytics for Risk Scoring and Treatment Recommendations

This system combines patient records, wearable readings, laboratory results and appointment information. A hybrid big data architecture is therefore required.

1. Hybrid Storage

- Relational databases store structured patient, appointment and laboratory records.
- Data lakes store medical documents and other unstructured information.
- NoSQL databases can store high-volume wearable readings.
- Object storage can store large medical files.

2. ETL and Integration

ETL pipelines extract data from hospitals and clinics, transform it into common formats and load it into the analytics platform.

Healthcare interoperability can be supported through **HL7/FHIR APIs**.

3. Real-Time Monitoring

- Wearable data is continuously sent to a streaming platform.
- Kafka can ingest the data.
- Flink/Spark Streaming analyzes the readings.
- Abnormal values can generate real-time alerts.

4. Distributed Processing
Frameworks such as **Apache Spark** can process millions of patient records across multiple nodes.

They can perform:

- Population-level statistics
- Disease trend analysis
- Risk analysis
- Treatment outcome analysis

5. Machine Learning
ML models can generate:

- Diagnosis risk scores
- Readmission risk
- Disease progression predictions
- Treatment recommendations
- Patient deterioration alerts

6. Encryption

- Data should be encrypted during transmission using secure protocols.
- Stored data should also be encrypted.
- Encryption keys should be securely managed.

7. Compliance and Governance
The system should implement:

- Role-based access
- Authentication
- Consent management
- Audit trails
- Data masking
- Data retention policies
- Privacy controls

8. Model Governance
Healthcare ML models should be monitored for accuracy, bias and reliability. Predictions should support healthcare professionals rather than blindly replacing clinical judgment.